

데이터 기반 의사결정 프로젝트 · 5강 실습

판단 기준 설계 & 가중치 매트릭스

빈칸 채우기 → 스스로 구현 → 도전 과제



빈칸 채우기



스스로 구현



도전 과제



2시간 · Python

오늘 실습 구성

1

가중치 매트릭스

🕒 55분

- STEP 0: 데이터 준비 (완성 코드)
- STEP 1: 가중치 직접 설계 ✎
- STEP 2: 가중 합산 직접 구현 ✎
- STEP 3: 역점수 처리 ✎
- 스스로 ①② + 도전 🚀

2

이해관계자 분석

🕒 35분

- STEP 1: 이해관계자 가중치 설계 ✎
- STEP 2: 비교 루프 구현 ✎
- 스스로 ③ + 도전 🚀

3

시각화

🕒 20분

- STEP 1: 막대 그래프 빈칸 ✎
- 스스로 ④ 레이더 차트 ✎
- 스스로 ⑤ 히트맵 ✎



미니 프로젝트

🕒 10분

- 새 데이터 전달
- 요구사항 5가지
- 코드 없이 직접 구현 ✎

따라하기 1

가중치 매트릭스로 최선의 선택 찾기

데이터 준비 → 가중치 설계 → 함수 구현 → 역점수 처리

🕒 55분

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt, matplotlib
matplotlib.rc('font', family='AppleGothic') # Mac → 'Malgun Gothic' (Windows)
plt.rcParams['axes.unicode_minus'] = False

# — OTT 오리지널 콘텐츠 평가 데이터 —
# 시청률·ROI·브랜드점수: 높을수록 유리 / 제작비(억): 낮을수록 유리
data = {
    '시청률': [22, 18, 15],
    'ROI': [1.8, 3.2, 2.5],
    '브랜드점수': [6, 7, 9],
    '제작비': [35, 18, 42],
}
df = pd.DataFrame(data, index=['콘텐츠A', '콘텐츠B', '콘텐츠C'])
print(df)
print('\n데이터 타입:', df.dtypes.to_dict())
```



예상 출력 (print 결과)

	시청률	ROI	브랜드점수	제작비
콘텐츠A	22	1.8	6	35
콘텐츠B	18	3.2	7	18
콘텐츠C	15	2.5	9	42



확인 포인트

제작비는 낮을수록 좋은 기준 →
그냥 가중치를 곱하면 안 됩니다! (STEP 3에서 처리)

✏️ 항색 ____ 를 직접 채우세요

```
# 판단 기준별 가중치 설정
# 조건: 합계가 반드시 1.0 이어야 함
weights = {
    '시청률': _____, # 0~1 사이
    'ROI': _____,
    '브랜드점수': _____,
    '제작비': _____, # 나중에 역점수로 바꿀 예정
}

# 합계 확인 (반드시 1.0이 되어야 함)
total = sum(_____)
print(f'가중치 합계: {total:.2f}')
assert _____ == 1.0, '합계 오류!'

# 가중치를 Series로 변환
wt = pd.Series(_____)
print('\n가중치 Series:', wt.to_dict())
```

🧠 가중치 설정 전 생각해보기

- Q1** 우리 OTT 플랫폼의 현재 전략적 목표는 무엇 인가요?
- Q2** 가장 중요한 기준과 덜 중요한 기준을 순서 매겨보세요
- Q3** 합계 1.0을 만족하면서 우선순위를 반영할 수 있나요?

💡 참고 예시 (보기 전에 먼저 시도!)

시청률 0.35 / ROI 0.25 / 브랜드 0.20 / 제작비 0.20

✎ 함수 본체를 완성하세요 — 힌트를 보고 ____ 를 채우세요

```
def weighted_score(df_scores, criteria_weights):  
    # df_scores: 기준별 DataFrame / criteria_weights: 딕셔너리  
    wt = pd.Series(____) # 딕셔너리를 Series로  
    cols = ____ # 가중치에 있는 컬럼만 선택  
    scores = df_scores[cols].mul(____) # 가중치 곱하기  
    return scores.sum(____) # 행 방향 합산  
  
# — 실행 —  
df['종합점수'] = weighted_score(df[['시청률', 'ROI', '브랜드점수', '제작비']], weights)  
df['순위'] = df['종합점수'].rank(ascending=____).astype(int)  
  
result = df.sort_values('종합점수', ascending=False)  
print(result[['시청률', 'ROI', '브랜드점수', '제작비', '종합점수', '순위']])
```

힌트 ①

pd.Series(dict) → dict의 키·값
으로 Series 생성

힌트 ②

list(weights.keys()) → 딕셔너리
키를 리스트로

힌트 ③

.mul(wt) → 각 열에 같은 이름
의 가중치를 곱함

힌트 ④

.sum(axis=1) → 행 방향(컬럼
들) 합산

⚠ 왜 역점수가 필요한가?

가중치를 곱할 때 '높을수록 좋다'고 가정합니다. 하지만 제작비는 낮을수록 유리 → 점수를 뒤집어야 합니다.

✍ 두 가지 방법을 직접 구현하세요

```
# 방법 A: 최댓값에서 빼기 (단순)
df['제작비역A'] = df['제작비'].max() - _____

# 방법 B: 비율 역점수 (0~10 정규화)
# 공식: 10 - (값 / 최댓값) * 10
df['제작비역B'] = 10 - (df['제작비'] / _____) * 10

print(df[['제작비', '제작비역A', '제작비역B']])

# — 역점수 포함 가중치로 재계산 —
weights_v2 = {
    '시청률': _____,
    'ROI': _____,
    '브랜드점수': _____,
    '제작비역B': _____, # 이제 제작비역 사용!
}

df['종합점수v2'] = weighted_score(df, _____)
print(df[['종합점수', '종합점수v2']].round(2))
```

📊 제작비 역점수 비교

콘텐츠	제작비	역A	역B(예상)
A	35억	7	1.7
B	18억	24	5.7
C	42억	0	0

💡 결과를 확인하세요

역점수 적용 전후로
종합점수 순위가 달라지나요?
어느 콘텐츠의 점수가 가장 크게 변했나요?

 미션

현재 설정으로는 특정 콘텐츠가 1위입니다. 가중치를 바꿔서 나머지 두 콘텐츠가 각각 1위가 되도록 만들어 보세요. 단, 가중치 합계 1.0은 반드시 유지해야 합니다.

 직접 코드를 작성하세요

```
# — 시나리오 1: 콘텐츠A를 1위로 만들기 —
w_scenA = {
    '시청률': ____,
    'ROI': ____,
    '브랜드점수': ____,
    '제작비역B': ____,
}
df['점수_A우선'] = weighted_score(df, w_scenA)
print('시나리오1 - A우선:', df['점수_A우선'].idxmax())

# — 시나리오 2: 콘텐츠C를 1위로 만들기 —
w_scenC = {
    '시청률': ____,
    'ROI': ____,
    '브랜드점수': ____,
    '제작비역B': ____,
}
df['점수_C우선'] = weighted_score(df, w_scenC)
print('시나리오2 - C우선:', df['점수_C우선'].idxmax())
```

 분석 질문 — 코드 실행 후 답해보세요

- ① 콘텐츠A를 1위로 만들려면 어떤 기준을 높여야 했나요? 그 이유는?
- ② 콘텐츠C는 어떤 기준에서 가장 강점이 있나요?
- ③ 콘텐츠B는 어떤 가중치 설정에서도 1위가 될 수 있나요?
- ④ 같은 데이터인데 가중치에 따라 결론이 달라지는 것을 어떻게 느꼈나요?

 미션

스케일이 다른 기준들(시청률 15-22, ROI 1.8-3.2, 브랜드 6-9)을 0~1 범위로 정규화한 뒤 가중 합산 점수를 비교하세요.

 직접 구현

```
# Min-Max 정규화 함수 작성
def minmax(series):
    """최솟값→0, 최댓값→1로 변환"""
    mn, mx = series.____(), series.____()
    return (series - ____ ) / ( ____ - mn)

# 제작비역 포함 컬럼에 적용
cols_to_norm = ['시청률', 'ROI', '브랜드점수', '제작비역B']
df_norm = df[cols_to_norm].apply(____)
print('정규화 결과:'); print(df_norm.round(3))

# 정규화 후 가중 합산
weights_norm = {'시청률':____, 'ROI':____,
                '브랜드점수':____, '제작비역B':____}
df['정규화점수'] = df_norm.mul(pd.Series(____)).sum(1)
print(df[['중합점수v2', '정규화점수']].round(3))
```

 도전 과제

가중치 최적화 탐색

특정 콘텐츠를 1위로 만들 수 있는
'모든' 가중치 조합을 찾아보세요.

```
# 가중치 조합 탐색 (브루트 포스)
results_search = []
for w1 in range(10, 61, 5): # 시청률 10~60%
    for w2 in range(10, 61, 5): # ROI
        for w3 in range(5, 41, 5): # 브랜드
            w4 = 100 - w1-w2-w3 # 나머지
            if w4 < 5: continue
            w = {'시청률': w1/100,
                'ROI': w2/100,
                '브랜드점수': w3/100,
                '제작비역B': w4/100}
            sc = weighted_score(df, w)
            winner = sc.idxmax()
            results_search.append(
                {'weights': w, 'winner': winner})

sr = pd.DataFrame(results_search)
print(sr.groupby('winner').size())
# 각 콘텐츠가 1위인 조합 수는?
```

따라하기 2

이해관계자별 관점 분석 — 같은 데이터, 다른 결론

가중치를 직접 설계하고, 루프로 이해관계자별 결론을 비교합니다

 35분

광고팀

시청률로 광고 단가 결정
→ 얼마나 많이 봤는가가 핵심

재무팀

투자 대비 수익이 최우선
→ ROI와 제작비가 핵심

경영진

장기 브랜드 가치와
종합적 포트폴리오 판단

👉 각 이해관계자의 가중치를 ____ 에 채워 넣으세요 (행 합계 = 1.0)

광고팀					1.0
재무팀					1.0
경영진					1.0

위 표를 기반으로 아래 딕셔너리를 완성하세요 (다음 슬라이드에서 사용)

```
stakeholders = {'광고팀': {'시청률': ____, 'ROI': ____, '브랜드점수': ____, '제작비역B': ____}, '재무팀': {...}, '경영진': {...}}
```

___ 를 채워서 루프를 완성하세요

```
# STEP 1에서 만든 stakeholders 딕셔너리 사용
comp_scores = {}

for sh_name, sh_weights in stakeholders.____(): # ①
    # 역점수 포함 컬럼만 사용
    wt = pd.Series(_____) # ②
    score = df[_____].mul(wt).sum(axis=1) # ③
    comp_scores[_____] = _____ # ④

comp_df = pd.DataFrame(_____) # ⑤
print('=== 이해관계자별 점수 ==='); print(comp_df.round(2))

# 각 이해관계자의 최선 선택
print('\n=== 각 이해관계자의 선택 ===')
for sh in comp_df.columns:
    winner = comp_df[sh]._____() # ⑥ idxmax
    score = comp_df[sh]._____() # ⑦ max
    print(f'{sh}: {winner} ({score:.2f}점)')

# 결론이 같은 이해관계자가 있나요?
print('\n선택이 일치하는가?', comp_df.____() == comp_df.____())
```

🔑 빈칸 힌트

- ① .items() → 딕셔너리를 key-value 쌍으로 순회
- ② sh_weights 를 Series로 변환
- ③ list(sh_weights.keys()) 로 컬럼 선택
- ④ 딕셔너리에 이해관계자명: 점수 저장
- ⑤ comp_scores 딕셔너리로 DataFrame 생성
- ⑥ .idxmax() → 최대값의 인덱스(이름) 반환
- ⑦ .max() → 최대값 반환

미션

'투자팀'을 새로운 이해관계자로 추가하세요.

투자팀은 ROI와 제작비 대비 효율성을 최우선으로 봅니다.

가중치를 직접 설계하고 기존 3명과 비교 결과를 출력하세요.

직접 작성하세요

투자팀 가중치 설계

```
stakeholders['투자팀'] = {
    '시청률': _____,
    'ROI': _____, # 높게 설정 권장
    '브랜드점수': _____,
    '제작비역B': _____, # 비용 효율 중요
}
```

비교 루프를 다시 실행하세요

(STEP 2 코드를 그대로 재실행하면 됩니다)

확인 질문: 투자팀의 선택이

재무팀과 같나요, 다른가요?

도전 과제

의견 충돌 시뮬레이션

이해관계자들의 의견이 충돌할 때, 어떻게 하나의 결론으로 수렴할 수 있을까요?

아래를 코드로 구현해보세요.

```
# 방법 1: 이해관계자 가중치 평균
avg_weights = {}
for crit in list(stakeholders.values())[0].keys():
    avg_weights[crit] = np.mean(
        [sh[crit] for sh in stakeholders.values()])
df['앙상블점수'] = weighted_score(df, avg_weights)
print('평균가중치 1위:', df['앙상블점수'].idxmax())

# 방법 2: 보르다 계산법 (순위 합산)
borda = pd.Series(0, index=df.index)
for sh_name in comp_df.columns:
    borda += comp_df[sh_name].rank(ascending=True)
print('보르다 점수:')
print(borda.sort_values(ascending=False))
# 앙상블 vs 보르다 - 결론이 같은가?
```

따라하기 3

결과 시각화

막대 그래프 빈칸 채우기 → 레이더 차트 직접 구현 → 히트맵 도전

 20분

주석 ①~⑦ 자리를 채우세요

```
fig, ax = plt.subplots(figsize=(10, 5))
x = np.arange(len(comp_df.____)) # ① 메뉴 수만큼 위치
bw = 0.25 # 막대 너비
colors = ['#F4845F', '#2196F3', '#9C27B0', '#FF9800'] # 이해관계자 수만큼

for i, (sh, c) in enumerate(zip(comp_df.columns, colors)):
    bars = ax.bar(x + i * _____, comp_df[sh], bw, # ②
                  label=_____, color=c, alpha=0.85) # ③ 범례 레이블
    ax.bar_label(bars, fmt='_____', padding=3, fontsize=9) # ④ 소수점 2자리

ax.set_xticks(x + bw * (_____ / 2)) # ⑤ 막대 중앙
ax.set_xticklabels(comp_df._____, fontsize=12) # ⑥ 콘텐츠명
ax.set_title('_____', fontsize=14, fontweight='bold') # ⑦ 원하는 제목
ax.set_ylabel('종합 점수', fontsize=11)
ax.legend(title='이해관계자', fontsize=10)
ax.grid(axis='y', alpha=0.4, linestyle='--')
ax.set_ylim(0, max(comp_df.max()) * 1.15)
plt.tight_layout()
plt.savefig('bar_chart.png', dpi=150) # 저장
plt.show()
```

💡 완성 후: `python3 hw5.py` 실행 → `bar\_chart.png` 저장 확인. 두 이해관계자의 선택이 일치하는가?

🔗 미션: 레이더 차트로 콘텐츠 A·B·C의 강점/약점 프로파일을 한 번에 비교하세요 (정규화된 값 사용 권장)

📐 코드 구조 힌트 (직접 채워서 완성)

```
labels = _____ # 기준명 리스트 (4개)
N = len(_____)
angles = np.linspace(0, 2*np.pi, N,
                    endpoint=False).tolist()
angles += angles[:1] # 마지막에 첫 값 추가 (폐곡선)

fig, ax = plt.subplots(figsize=(6,6),
                        subplot_kw=dict(polar=True))
colors = ['#F4845F', '#2196F3', '#FF5722']

for content, color in zip(df_norm.index, colors):
    vals = df_norm.loc[content, labels].tolist()
    vals += _____ # 폐곡선 처리
    ax.plot(_____, vals, 'o-', lw=2,
           color=color, label=content)
    ax.fill(_____, vals, alpha=0.1, color=color)

ax.set_xticks(_____)
ax.set_xticklabels(_____, fontsize=11)
ax.set_title('콘텐츠별 평가 프로파일', pad=20)
ax.legend(loc='upper right')
```

🔗 예상 결과 & 분석 포인트

A의 강점

시청률 방향으로 가장 멀리 뻗어나감

B의 강점

ROI + 제작비역에서 균형잡힌 프로파일

C의 강점

브랜드점수 방향으로 독보적

공통 약점

어느 콘텐츠도 모든 기준에서 1위가 아님

미션

comp_df (이해관계자별 점수 행렬)를
seaborn 히트맵으로 시각화하세요.

직접 작성하세요

```
import seaborn as sns

fig, ax = plt.subplots(figsize=(8, 4))

# 힌트: sns.heatmap(data, annot, fmt, cmap, ...)
sns.heatmap(
    data=____,          # comp_df 사용
    annot=____,         # 숫자 표시 여부
    fmt='____',         # 소수점 형식
    cmap='YlOrRd',      # 색상 팔레트
    linewidths=0.5,
    ax=ax
)
ax.set_title('____')   # 원하는 제목
ax.set_xlabel('____')
ax.set_ylabel('____')
plt.tight_layout()
plt.savefig('heatmap.png', dpi=150)
plt.show()
```

도전 과제

가중치 히트맵 + 순위 맵

이해관계자들의 '가중치 차이'를 히트맵으로 시각화하고, 점수
히트맵과 나란히 배치하세요.

```
# subplot 2개: 가중치 vs 점수 나란히
fig, axes = plt.subplots(1, 2,
                          figsize=(14, 4))

# — 왼쪽: 이해관계자별 가중치 히트맵 —
wt_df = pd.DataFrame(stakeholders).T
sns.heatmap(wt_df, annot=True, fmt='.2f',
            cmap='Blues', ax=axes[0])
axes[0].set_title('이해관계자별 가중치')

# — 오른쪽: 종합 점수 히트맵 —
sns.heatmap(comp_df.T, annot=True, fmt='.2f',
            cmap='YlOrRd', ax=axes[1])
axes[1].set_title('이해관계자별 종합점수')

plt.tight_layout()
plt.savefig('dual_heatmap.png', dpi=150)
plt.show()

# 두 히트맵 사이에 어떤 패턴이 보이나요?
```

 미니 프로젝트

배달 앱 카테고리 전략 메뉴 선정

코드 없이 처음부터 구현 · 힌트 슬라이드 없음 · 결과 팀끼리 비교

 10분

 스스로 구현 — 코드 미제공

데이터 — 배달 앱 카테고리별 성과 지표

미니 프로젝트

아래 데이터를 pandas DataFrame으로 직접 입력하세요

 치킨	82	22,000	58	4.3	12	4,200
 피자	45	28,000	42	4.1	18	2,800
 한식	120	13,000	65	4.5	8	7,500
 중식	38	18,000	48	4.0	15	1,900
 일식/스시	29	35,000	55	4.6	22	1,400

⚠ 주의 사항

영업이익률↑이 좋음 / 제휴음식점수↑이 좋음 / 객단가는 전략 방향에 따라 다름 (고급화→높을수록, 대중화→낮을수록)

요구사항 — 이것을 모두 구현하세요

미니 프로젝트

①

2분

DataFrame 생성

슬라이드 19의 데이터를 pandas DataFrame으로 만드세요.

객단가의 콤마(,)를 제거하고 숫자형으로 변환하는 것도 잊지 마세요.

②

2분

역점수 처리

어떤 기준이 '낮을수록 좋은가'를 판단하고, 해당 기준에 역점수 처리를 적용하세요.

(전략 방향에 따라 객단가 처리 방법이 달라질 수 있습니다)

③

2분

나만의 가중치로 Top 3 카테고리 도출

여러분이 배달 앱 CEO라면 어떤 카테고리에 집중하겠습니까?

가중치를 설계하고 `weighted_score()` 함수로 순위를 계산하세요.

④

2분

3가지 전략 시나리오 비교

① 수익성 극대화 ② 사용자 충성도 ③ 시장 점유율
각 시나리오별 가중치를 설계하고 1위 카테고리를 비교하세요.

⑤

2분

결과 시각화

3가지 시나리오 결과를 그룹 막대 그래프 또는 히트맵 중 하나로 시각화하세요.

파일로 저장까지 완료하세요 (savefig).

핵심 코드 레퍼런스 (막히면 참고!)

① 가중치 합산

```
df[cols].mul(pd.Series(weights)).sum(axis=1)
```

② 이해관계자 루프

```
for name, w in stakeholders.items():  
    scores[name] = df[cols].mul(pd.Series(w)).sum(1)
```

③ Min-Max 정규화

```
def minmax(s): return (s - s.min()) / (s.max() -  
s.min())  
df_norm = df[cols].apply(minmax)
```

④ 역점수 (비율)

```
df['비용역'] = 10 - (df['비용'] / df['비용'].max() * 10)
```

⑤ 막대 그래프

```
for i, (sh, c) in enumerate(zip(df.cols, colors)):  
    ax.bar(x+i*bw, df[sh], bw, label=sh, color=c)
```

⑥ 레이더 차트

```
angles = np.linspace(0, 2*pi, N, endpoint=False).tolist()  
ax.plot(angles+angles[:1], vals+vals[:1], 'o-')
```

오늘 실습에서 배운 것



가중치 설계는 '무엇을 중요하게 보느냐'를 수치로 표현하는 것



역점수 처리 없이 가중 합산하면 기준 방향이 잘못된 결론을 냄



이해관계자마다 가중치가 다르기 때문에 '같은 데이터, 다른 결론'이 나온다



시각화(막대 그래프·레이더·히트맵)는 결론을 더 설득력 있게 전달한다



Python으로 이 과정을 자동화하면 수백 가지 가중치 조합을 순식간에 비교할 수 있다